

TDDFT Refactor: Source-Code Mapping

Pre-refactor `RmgTddft.cpp` → Post-refactor `rmg::tddft` class

Companion to `rmg_tddft_theory.tex`

Jacek Jakowski (working notes, May 2026)

May 26, 2026

Abstract

In May 2026 the RMG-DFT upstream repository underwent a structural refactor that affected the TDDFT module. The pre-refactor monolithic free function `RmgTddft<O,M>(...)` was deleted; its functionality was redistributed into a new `rmg::tddft<OrbitalType,MatrixType>` class split across three source files. The annotated copy of the old file in the frozen `explore` branch serves as a *Rosetta Stone* for understanding what the new code does and where each piece of the old algorithm now lives.

This note documents the structural change, the section-by-section mapping between old and new code, the post-refactor status of the eight known theory/code gaps (`GAP-1...GAP-8`) catalogued in the companion theory document, and three substantive additions that the refactor introduced. It serves as an entry point for further work on non-collinear spin RT-TDDFT and Ehrenfest dynamics.

Contents

1	Context: why this document exists	1
2	Structural change	2
3	Section-by-section mapping	2
4	GAP audit (post-refactor)	3
5	New items introduced by the refactor	4
5.1	Per-call bootstrap (<code>first_step</code>)	4
5.2	Running time-averaged density (<code>trho</code>)	5
5.3	Ehrenfest force update (<code>TDDFT_CVE</code> branch)	5
6	Forward look	6

1 Context: why this document exists

The `explore` branch of the personal fork `jjakowski/rmgdft-sandbox` was used for several months as a sandbox for annotating the RT-TDDFT C++ source with detailed physics commentary. Three files were annotated: `TDDFT/RMG_TDDFT/RmgTddft.cpp`, `CurrentNlpp.cpp`, and `VecPmatrix.cpp`. The manuscript and theory notes also live on `explore`.

When the upstream sync was performed in May 2026, three issues appeared:

- `RmgTddft.cpp` was *deleted* from upstream and replaced by a different file structure — annotation could not be carried forward by simple copy.
- `CurrentNlpp.cpp` and `VecPmatrix.cpp` were unchanged upstream — annotation *could* be copied verbatim.
- `explore` also held months of manuscript work that must be preserved.

The resolution was to (i) tag the pre-sync state of `develop` as `pre-TDDFT-annotation`, (ii) leave `explore` frozen as a permanent Rosetta Stone, (iii) sync `develop` with upstream, and (iv) create a clean `explore-new` branch off the post-sync `develop`. Annotations from the unchanged files were migrated by direct copy (Task A). Annotations for `RmgTddft.cpp` were migrated by re-mapping each annotated section to its new location in the refactored files (Task B). This document captures the structural understanding developed during Task B.

2 Structural change

Before. A single templated free function did everything inline:

```
template <typename OrbitalType, typename MatrixType>
void RmgTddft( spinobj<double>& vxc,
               fgobj<double>& vh,
               fgobj<double>& vnuc,
               spinobj<double>& rho,
               fgobj<double>& rhocore,
               fgobj<double>& rhoc,
               Kpoint<OrbitalType>** Kptr );
```

1245 lines in one function: SETUP (memory, file open, `ReadData`) → velocity-gauge init / kick → ground-state J_0 baseline → outer time loop (STEP 1 / 2 / 3) → cleanup (file close, write checkpoint).

After. A C++ class `rmg::tddft<OrbitalType,MatrixType>` split across three files:

File	Members
<code>rmg_tddft.cpp</code>	constructor <code>tddft(...)</code> (setup + init + kick) method <code>tddft_md()</code> (outer time loop) destructor <code>~tddft()</code> (write + close + free) helpers <code>tstconv()</code> , <code>gather_rho_matrix()</code>
<code>rmg_tddft_energy.cpp</code>	methods <code>tddft_energy_init()</code> , <code>tddft_energy()</code>
<code>rmg_rotate_sint.cpp</code>	free function <code>rmg::rotate_sint<O>()</code> (Ehrenfest helper)

The split reflects a real architectural change, not just file reorganisation: in the new design `tddft_md()` can be invoked *multiple times* from an outer Ehrenfest MD loop, each invocation propagating the electrons for `ct.tddft_steps` steps before returning forces on the nuclei. The constructor sets up the persistent state once; the destructor flushes it once. In the old design every TDDFT call was a single self-contained run from setup to cleanup, with no notion of repeated invocations.

3 Section-by-section mapping

Line numbers in the rightmost column refer to TDDFT/RMG_TDDFT/`rmg_tddft.cpp` after the Task B annotation commit (a998da7c2). Line numbers in the leftmost column refer to the frozen `explore` version of `RmgTddft.cpp`.

Old section (RmgTddft.cpp)	New member	New location (rmg_tddft.cpp)
File-level header + audit (l. 23–152)	GAP-1.8 — (file header)	Top of file, after GPL header (l. 25–165)
Velocity-gauge initialization (l. 606–657)	constructor	Inside <code>if(ct.tddft_mode == VECTOR_POT)</code> block, l. 561
Inline STEP A / B / C labels (l. 663, 668, 677)	constructor	l. 615, 620, 629
Ground-state J_0 baseline (l. 696–707)	constructor	l. 648 (before <code>zdotc</code> loop populating <code>current0[]</code>)
Outer time loop comment (l. 794–803)	<code>tddft_md()</code>	l. 830 (before <code>for(int tddft_steps...)</code>)
STEP 1 — PREDICTOR (l. 815–820)	<code>tddft_md()</code>	l. 851 (before <code>extrapolate_Hmatrix</code>)
STEP 2 — CORRECTOR SCF (l. 849–854)	<code>tddft_md()</code>	l. 886 (before SCF <code>while</code>)
STEP 2a — Magnus operator (l. 885–889)	<code>tddft_md()</code>	l. 921 (before <code>magnus(...)</code>)
STEP 2b — BCH propagator (l. 892–898)	<code>tddft_md()</code>	l. 929 (before <code>eldyn.ort(...)</code>)
STEP 2c — Density update (l. 916–920)	<code>tddft_md()</code>	l. 955 (before <code>Rho RmgTimer</code>)
STEP 2d — Update potentials (l. 971–973)	<code>tddft_md()</code>	l. 1009 (before <code>compute_vxc(...)</code>)
STEP 2e — Update Hamiltonian (l. 991–999)	<code>tddft_md()</code>	l. 1029 (before <code>vtot</code> accumulation)
Convergence check (l. 1038–1042)	<code>tddft_md()</code>	l. 1078 (after <code>daxpy</code> , before <code>tstconv</code>)
STEP 3 — Advance (l. 1096–1101)	<code>tddft_md()</code>	l. 1144 (before advance <code>for</code> loop)
TIME-DEPENDENT CURRENT $J(t)$ (l. 1113–1130)	<code>tddft_md()</code>	l. 1160 (inside <code>if(VECTOR_POT)</code> branch)

All twelve annotation blocks were ported verbatim from the Rosetta Stone; only their position changed to follow the constructor / `tddft_md()` split.

4 GAP audit (post-refactor)

Each of the eight gaps documented in `rmg_tddft_theory.tex` was re-checked against the refactored code. *None was fixed by the refactor.* The evidence for each is summarised below.

GAP-1: Velocity-gauge mode is delta-kick only

The time-varying field update `daxpy(A_0*cos( $\omega t$ ), Pmatrix, Hmatrix_0)` is still present but *commented out* (`rmg_tddft.cpp` l. 838–846 inside `tddft_md()`). `ct.tddft_frequency` is read from input but has no effect on the propagation.

GAP-2: Diamagnetic current is missing

`current[]` is still computed as $J_\alpha = \text{Re Tr}(P P^\alpha)$ only — see l. 1175–1186 inside `tddft_md()`. The $-n(r, t) A(t)$ contribution is absent. For the delta-kick (where $A \rightarrow 0$ after $t = 0$) this is fine; if GAP-1 is fixed, GAP-2 must also be.

GAP-3: A^2 term is dropped

The minimal-coupling Hamiltonian $(p + A)^2/2 = p^2/2 + A \cdot p + A^2/2$ is truncated to the linear $A \cdot p$ term. No amplitude check enforces the linear-response regime in which this is valid.

GAP-4: Non-collinear spin in VECTOR.POT path is incomplete

The `vtot` update at STEP 2e uses only the scalar part of V_{xc} . Same code structure as before; both `// noncoll need change` comments are still present at l. 1039 and l. 1048 of `rmg_tddft.cpp`. This is directly relevant to the planned non-collinear spin RT-TDDFT work.

GAP-5: `Hmatrix_m1_cpu` triple-purposed

Same buffer reuse pattern as before: before SCF it is $H(t - dt)$; during STEP 2a it is overwritten with the Magnus operator Ω ; during STEP 2e it is overwritten with $\langle \delta V \rangle$. $H(t - dt)$ is restored only at STEP 3 via `memcpy(Hmatrix_m1_cpu, Hmatrix_0_cpu, ...)`.

GAP-6: `Hmatrix_cpu` is never explicitly reset to $H(t)$

Same convention as before: `Hmatrix_cpu` carries the converged H from the previous step and the SCF loop adds *incremental* potential corrections δV on top of it. Any future modification that breaks the “*vtot* is incremental” invariant will silently corrupt H .

GAP-7: No guard on mode vs. boundary conditions

No check that EFIELD is used only with finite molecules (`ct.is_gamma == true`) or that VECTOR.POT is used only with proper k-point sampling. Misuse compiles and runs but produces unphysical results.

GAP-8: `ct.efield.tddft.crd`s semantics differ between modes

The same input array is interpreted as E_0 (length gauge, EFIELD) or A_0 (velocity gauge, VECTOR.POT). The relationship $E_0 = A_0 \omega$ means the two modes require different input values for the same physical field strength. Not enforced in input parsing.

Implication. The refactor was a *structural* reorganisation, not a physics rewrite. Every theoretical limitation identified before the refactor remains an open item in `explore-new`.

5 New items introduced by the refactor

Three blocks of code have no analogue in the old `RmgTddft.cpp`. They are documented inline in the post-refactor source; the discussion below is for forward reference.

5.1 Per-call bootstrap (`first_step`)

Located near the top of `tddft_md()` (l. 776–800):

```
static int first_step;
for(int kpt = 0; kpt < ct.num_kpts_pe; kpt++)
{
    BetaProjector->project(...);    // ALWAYS: refresh sint = <beta/phi>
    if(first_step) {
        HSmatrix(...);              // rebuild H_ij = <phi_i|H_KS|phi_j>
        DistributeMatrix(...);      // scatter into BLACS storage
        // re-seed Hmatrix_m1/0/1 with fresh H
    }
}
first_step++;
```

The condition `if(first_step)` evaluates to `false` on the very first call (since the `static int` is zero-initialized) and `true` on every subsequent call. This skips the H rebuild on the first invocation — because the constructor already filled H with the diagonal of ground-state eigenvalues, which is valid as long as ions have not moved. From the second call onward (i.e., after the outer MD loop has moved ions), the ground-state diagonal H is no longer valid and must be rebuilt from scratch via `HSmatrix` in the current geometry.

Implication for non-collinear spin RT-TDDFT

For non-collinear spinor wavefunctions the bootstrap path matters because `HSmatrix` must produce the correct 2×2 spin-space matrix elements. Verify that the `HSmatrix` implementation in `TDDFT/RMG_TDDFT/HSmatrix.cpp` respects `ct.noncoll_factor` before enabling `noncoll` with multi-step Ehrenfest dynamics.

5.2 Running time-averaged density (`trho`)

After each propagation step (`rmg_tddft.cpp` l. 1104–1111):

$$\mathbf{trho}(\mathbf{r}) += \rho(\mathbf{r}, t_n)$$

After the outer time loop completes, `trho` is rescaled by $1/N_{\text{steps}}$:

$$\bar{\rho}(\mathbf{r}) = \frac{1}{N_{\text{steps}}} \sum_{n=0}^{N_{\text{steps}}-1} \rho(\mathbf{r}, t_n).$$

The time-averaged density $\bar{\rho}$ is the input to the `Force(...)` call in the Ehrenfest block (Section 5.3), rather than the final-snapshot density $\rho(t_{\text{final}})$.

Why averaging? The instantaneous TD density carries fast electronic oscillations (interband and plasmon-like) that, if used directly, would produce noisy nuclear forces and unstable trajectories. Averaging over the propagation window damps these oscillations and provides smoother forces. This is conceptually similar to the “time-averaged forces” used in adiabatic-following schemes in quantum-chemistry molecular dynamics codes; physically it is a low-pass filter on $F[\rho(t)]$.

5.3 Ehrenfest force update (TDDFT_CVE branch)

The post-refactor `tddft_md()` ends with an Ehrenfest force-update block (l. 1259–1305 in `rmg_tddft.cpp`). It runs when `ct.forceflag == TDDFT_CVE`, i.e., when the user has requested Combined-Velocity-Verlet/Ehrenfest dynamics. The algorithm:

1. **Rotate force pointers.** `Atoms[ion].RotateForces()` shifts old per-ion force snapshots back by one slot so the Beeman/Velocity-Verlet integrator has access to $F(t - dt)$, $F(t - 2dt)$ after the new force is written.
2. **Gather global density matrix.** `gather_rho_matrix(global, Pn0)` collects the BLACS-distributed $P(t_{\text{final}})$ into a global $N_{\text{states}} \times N_{\text{states}}$ matrix on each rank.
3. **Rotate sint into TD basis.** `save_sint()` preserves the ground-state `sint` = $\langle \beta_{lm} | \phi_j \rangle$; occupations are temporarily set to 1.0 (slot [3] holds the ground-state values for restore); `rmg::rotate_sint(...)` multiplies the ground-state `sint` by the density matrix to produce the TD-basis `sint`. Occupations f_j are set to unity because the per-orbital weights are already encoded in P_{ij} .
4. **Compute forces from time-averaged density.** `Force(trho.up, trho.dw, ..., Kptr)` computes the nuclear forces using $\bar{\rho}$, not the final snapshot.
5. **Restore state.** `restore_sint()`, restore occupations, then `get_ddd(...)` refreshes the D_{ij} dual-projector coefficients for the next call.

Open: nonlocal-PP contribution to the Ehrenfest force

The current `Force(...)` call assumes the local potential contribution is the dominant force component. For norm-conserving pseudopotentials, the nonlocal contribution to the Hellmann–Feynman force is non-zero and depends on the TD density via the same projectors used in `CurrentNlpp.cpp`. Active development on this lives in the `develop-Ehrenfest` branch. The current-operator analogue $i[V_{\text{NL}}, \mathbf{r}]$ in `CurrentNlpp.cpp` is the relevant reference; the force analogue is $\frac{\partial}{\partial R_I} \langle \phi | V_{\text{NL}} | \phi \rangle$ contracted with $P(t)$.

6 Forward look

The Task A/Task B migration unblocks three lines of development:

1. **Non-collinear spin RT-TDDFT.** Requires fixing GAP-4 (`vtot` must include the off-diagonal spin components of V_{xc}) in `tddft_md()` l. 1039–1048. The two existing `// noncoll need change` markers point to the exact lines. Also verify the bootstrap (Section 5.1) handles the 2×2 spin block correctly.

2. **Ehrenfest dynamics with NL-PP forces.** Open issue flagged in Section 5.3. The `develop-Ehrenfest` branch is the active worktree for this work; the relevant reference files in this branch are `Force/Force.cpp` (ground-state version) and `TDDFT/RMG_TDDFT/CurrentNlpp.cpp` (current-operator analogue, annotated).
3. **Spin + Ehrenfest combined.** Requires both of the above. The `rmg::tddft` class is already structured to support it: `tddft.md()` is designed to be re-entrant from an outer MD loop, and the `first_step` bootstrap rebuilds H in the current geometry on every call.

Pre-existing physics work that is still relevant. GAP-1 (sustained field) and GAP-2 (diamagnetic current) are the two remaining items needed to extend RT-TDDFT from delta-kick optical-spectra calculations to true pump-probe and laser-driven simulations. Both require changes in `rmg_tddft.cpp` at the locations identified in Section 4.

Document provenance

- Frozen Rosetta Stone: `git show explore:TDDFT/RMG_TDDFT/RmgTddft.cpp`
- Task A commit (CurrentNlpp + VecPmatrix): `b01af2025`
- Task B commit (`rmg_tddft.cpp` annotations): `a998da7c2`
- Pre-sync anchor tag: `pre-TDDFT-annotation`
- Companion theory document: `docs/rmg_tddft_theory.tex` (in the frozen `explore` branch)